

SALESTORM — AI Usage Note / Prompt Summary

1. Purpose of AI Usage

AI tools were used as engineering accelerators, not as replacements for system-design decisions.

The team first analyzed the SALESTORM requirements, concurrency constraints, inventory consistency requirements, payment workflow and reliability scenarios. AI was then used to accelerate documentation, prototype development, testing and design artefact generation.

Primary principle:

Design First → AI-Assisted Implementation → Validation → Technical Defense

2. AI Tools Used

AI Tool	Purpose
-----	-----
Antigravity	Frontend/backend prototype development and implementation assistance
Claude	Presentation/documentation generation and content organization
ChatGPT	Architecture reasoning, API design, UML/diagram assistance, testing scenarios and technical documentation

3. Areas Where AI Was Used

A. Prototype Development

AI was used to accelerate development of:

- * Frontend dashboard
- * Backend API structure
- * Inventory APIs

- * Reservation APIs
- * Payment workflow
- * Order workflow
- * Simulation interfaces
- * Failure-scenario demonstrations

AI-generated implementation was reviewed and tested by the team before inclusion.

B. Concurrency Simulation

AI was used to help create test/simulation logic for:

10,000 concurrent purchase requests



100 available units



Concurrent inventory reservation



Expected successful reservations ≤ 100



OVERSOLD = 0

The simulation was used as ****validation evidence****, not as a replacement for the architecture.

C. Test Cases

AI assisted in generating test scenarios covering:

- * Successful reservation
- * Out-of-stock requests
- * Duplicate requests

- * Reservation expiry
- * Payment success
- * Payment failure
- * Payment timeout
- * Order Service failure
- * Payment success + Order Service recovery
- * Idempotent retries

The generated tests were reviewed and executed by the team.

4. Documentation Assistance

AI was used to accelerate creation of:

- * Requirements & Assumptions Document
- * HLD documentation
- * LLD documentation
- * API specifications
- * Database design documentation
- * SOLID mapping
- * Design Pattern mapping
- * Architecture Decision Records
- * Scalability & Reliability Design
- * Security & Observability Design
- * UML/sequence/state diagram drafts
- * Presentation content

The team reviewed the generated documentation and aligned it with the final architecture.

5. Example AI Prompt Used for Prototype Development

Master Development Prompt

Design and implement a prototype for SALESTORM, a high-scale e-commerce flash-sale system.

Scenario:

10,000 customers simultaneously attempt to purchase Product X, which has only 100 available units.

The system must guarantee:

1. Zero inventory overselling.
2. Idempotent reservation and payment requests.
3. Temporary inventory reservations with expiry.
4. Reliable payment processing.
5. Reliable order creation.
6. Recovery from Payment Service and Order Service failures.
7. Horizontal scalability of stateless services.
8. Clear separation of service responsibilities.

Use atomic conditional database updates for inventory reservation.

The inventory operation must follow:

UPDATE inventory

SET available_quantity = available_quantity - 1

WHERE product_id = ?

AND available_quantity >= 1;

Use idempotency keys for reservation, payment and order operations.

Use a transactional outbox for reliable event publishing.

Include retry, timeout, circuit-breaker and reservation-expiry behaviour where appropriate.

Create a clean frontend dashboard demonstrating:

- System overview
- Flash-sale traffic
- Inventory state
- Concurrency simulation
- Reservation results
- Payment state
- Order state
- Failure recovery
- System health

The implementation must remain consistent with the architecture.

Do not introduce unnecessary complexity.

Generate clean, modular and explainable code.

6. Example AI Prompt for Concurrency Testing

Create a concurrency test for SALESTORM.

Given:

- Initial inventory = 100
- Concurrent purchase requests = 10,000

Execute concurrent reservation attempts against the inventory API.

Verify:

1. Successful reservations ≤ 100 .
2. Inventory never becomes negative.
3. Oversold count = 0.
4. Duplicate idempotency keys do not create duplicate reservations.
5. Failed requests receive an appropriate response.

6. Final inventory state is consistent.

Report:

- Total requests
- Successful reservations
- Failed reservations
- Duplicate requests
- Final available inventory
- Oversold count
- Execution time

7. Example AI Prompt for Failure Testing

Simulate the following SALESTORM failure scenario:

1. Order Service becomes unavailable.
2. Customer payment succeeds.
3. PaymentSucceeded event is written to the transactional outbox.
4. Order Service remains unavailable for 30 seconds.
5. Order Service recovers.
6. Message broker retries the event.
7. Order Service processes the event idempotently.
8. Order is successfully created.

Verify that:

- Payment is not duplicated.
- Order is not duplicated.
- The event is not lost.
- The reservation remains consistent.
- The final order reaches CONFIRMED state.

8. AI Validation Process

AI-generated output was not directly accepted without review.

The validation process was:

Business Requirements



Architecture Design



AI-Assisted Generation



Code / Diagram Review



Testing



Failure Scenario Validation



Architecture Revision



Final Demonstration

9. Human Ownership

The team remains responsible for:

- * Architecture selection
- * Service boundaries
- * Inventory consistency
- * Concurrency strategy
- * Database decisions

- * Payment workflow
- * Order lifecycle
- * Failure recovery
- * Security decisions
- * Scalability strategy
- * Trade-off analysis
- * Final technical explanation

AI-generated components were treated as implementation assistance, not as authoritative design decisions.

10. Key AI-Assisted Validation Results

The prototype/simulation was designed to validate the most important SALESTORM assumptions:

Scenario	Expected Result
-----	-----
10,000 requests / 100 units	Maximum 100 successful reservations
Inventory reservation	No negative inventory
Duplicate Buy request	Existing result returned
Payment failure	Reservation released
Reservation timeout	Stock released
Payment timeout	Reconciliation performed
Payment success + Order failure	Event retained and retried
Order Service recovery	Order eventually created
Repeated event	Idempotent processing
Inventory integrity	**OVERSOLD = 0**

11. AI Usage Boundaries

AI was not used to replace:

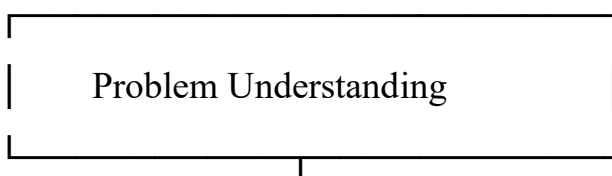
- * Requirement analysis
- * Architecture selection
- * Concurrency reasoning
- * Inventory consistency decisions
- * Payment state modelling
- * Order state modelling
- * Security architecture
- * Reliability strategy
- * Architecture trade-off decisions
- * Final jury defense

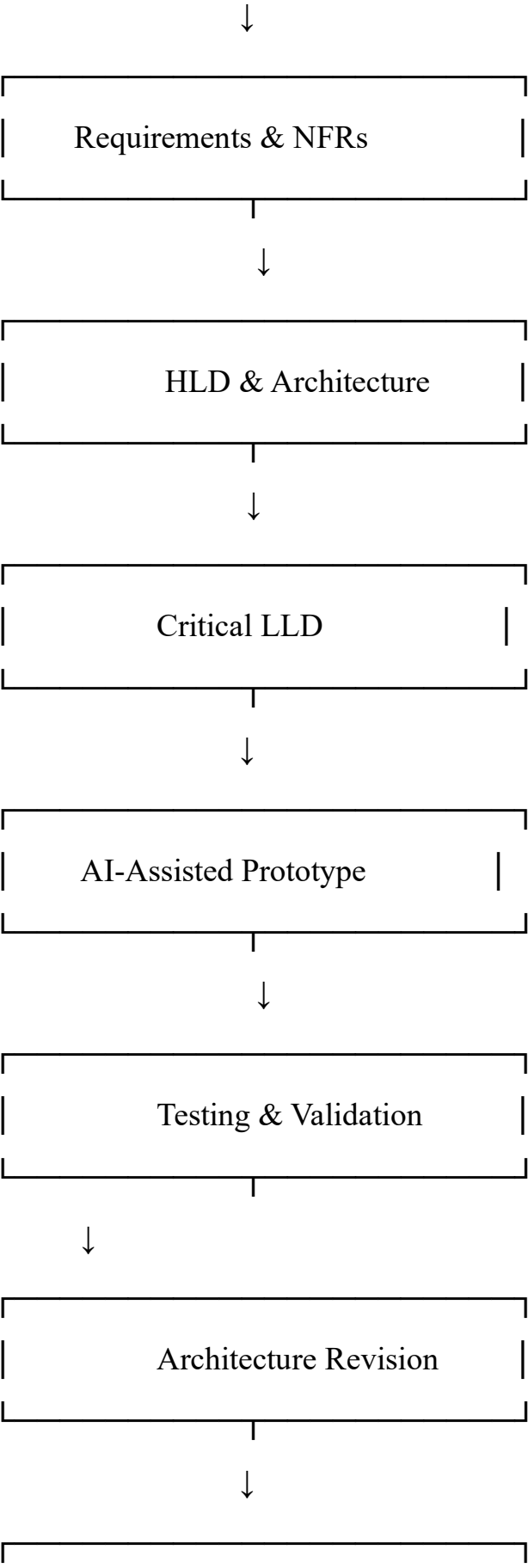
These decisions were made based on the SALESTORM requirements and engineering reasoning.

12. Transparency Statement

AI tools were used throughout SALESTORM as productivity and engineering-assistance tools for prototype implementation, test generation, documentation, UML drafts and simulation development. The core architecture, service boundaries, concurrency strategy, inventory consistency mechanism, payment/order workflow, reliability model and architectural trade-offs were reviewed and owned by the team. All AI-assisted artefacts were validated against the defined requirements before demonstration.

AI-Assisted Workflow





	Final Technical	
	Defense	

Final Principle

AI accelerated the implementation; engineering judgment determined the architecture.